Ders 19 Çalışma Özeti

Ders 19 Çalışma Özeti

Genel Konular

- Sıralama algoritmaları (sorting)
  - Sıralama algoritmalarıyla bundan sonra çok sık karşılaşılacak. Bir işi yapmadan önce veriyi sıralamak, mantıklıysa tercih edilebilir.
  - Bu derste bubble sort tekrar edilir ve iyileştirilir, selection sort anlatılır, insertion sort'un mantığı tanıtılır.
- Bubble sort tekrarı (ekrandan, 00:32:30)
  - Yan yana iki eleman karşılaştırılır. Küçükten büyüğe sıralamada soldaki büyükse yer değiştirilir.
  - Örnek `9 1 3 2 4 7 8 6`. İlk tur (first round, iç döngü): 9 her karşılaştırmada sağa geçer ve tur sonunda `1 3 2 4 7 8 6 9` elde edilir. En büyük eleman en sona itilmiştir.
  - İkinci tur (second round): `1 2 3 4 7 6 8 9`. Dizinin en büyük ikinci elemanı (8) yerine yerleşir.
  - Akış diyagramı: dış döngü `I ← 1 || 1 || N-1`, iç döngü `J ← 1 || 1 || N-I`. `Dizi[J] > Dizi[J+1]` ise `tmp = Dizi[J]`, `Dizi[J] = Dizi[J+1]`, `Dizi[J+1] = tmp`.
  - İç döngünün sonu `N-I`'dir, çünkü her turda sondaki bir eleman daha yerine oturur ve tekrar karşılaştırılmasına gerek yoktur.
  - Dış döngü N−1 kez döner. N−1 eleman yerine yerleşince sonuncusu da kendiliğinden yerindedir.
  - C'de diziler 0'dan başladığı için iç döngü sınırı `N-I-1` olur. Bir öğrenci C kodunda bu sınırla doğru sonuç aldığını söyledi. Hoca bunun normal olduğunu, uç değer kontrollerinin her zaman yapılması gerektiğini vurguladı.
- Bubble sort'un karmaşıklığı
  - En kötü durumda (worst case, ör. `9 8 7 6 4 3 2 1`) karşılaştırma sayısı $(N-1)\times(N-I)$ şeklinde ifade edildi. Toplam $(N-1)+(N-2)+\cdots+1=\frac{N(N-1)}{2}$ karşılaştırmadır.
  - Algoritma karmaşıklığı, en yüksek üslü terimle ifade edilir. Küçük üslü terimler atılır. Örneğin N^2-2N+3 için karmaşıklık N^2'dir, N^3 olsaydı N^2 de atılırdı. Bubble sort'un en kötü durum karmaşıklığı N^2'dir.
  - Bu hâliyle en iyi durumda (best case, dizi zaten sıralı, ör. `1 2 3 4 6 7 8 9`) da N^2 karşılaştırma yapılır. Bilgisayar dizinin sıralı olduğunu bilmediği için bütün karşılaştırmaları yapar, ama hiçbir yer değiştirme olmaz.
  - Bubble sort'un en zayıf noktası çok sayıda yer değiştirme yapmasıdır. Her yer değiştirme belleğe erişim, okuma ve yazma demektir ve maliyetlidir.
- İyileştirilmiş bubble sort
  - Fikir: bir turda hiç yer değiştirme olmazsa dizi sıralanmış demektir ve dış döngü durdurulabilir.
  - Dış döngü for yerine while olur. Bir `flag` değişkeni kullanılır:
    - Başta `flag = 1` olur ki döngüye girilsin.
    - Her turun başında, iç döngüye girmeden `flag = 0` yapılır.
    - İç döngüde if'in true koluna girilirse (yer değiştirme olursa) `flag = 1` yapılır.
    - Dış while, `flag = 1` olduğu sürece devam eder.
  - En iyi durumda (sıralı dizi) ilk turda hiç yer değiştirme olmaz ve döngü biter. Karşılaştırma sayısı N mertebesinde olur. En kötü durumda yine N^2'dir.
  - Örnek: `2 1 3 4 6 7 8 9` dizisinde ilk turda yalnızca 1 ile 2 yer değiştirir. İkinci turda hiç değişiklik olmaz ve sıralama biter.
- Selection sort (ekrandan, 01:16:20)

  - Mantık: her turda kalan elemanların en küçüğü bulunur ve sıradaki konuma yerleştirilir.
  - Örnek `5 1 3 2 4 8 0`: ilk turda en küçük eleman (0) başa gelir. İkinci tur ikinci elemandan başlar ve kalanların en küçüğü (1) ikinci konuma gelir. Böyle devam eder.
  - İlk (ilkel) versiyon: dış döngü `I ← 1 || 1 || N−1`, iç döngü `J ← I+1 || 1 || N`. `Dizi[I] > Dizi[J]` ise hemen yer değiştirilir. Böylece bir turda birden fazla yer değiştirme olabilir.
  - İyileştirilmiş versiyon:
    * Dış döngü `I ← 1 || 1 || N−1`. Her turun başında `min = I`.
    * İç döngü `J ← I+1 || 1 || N`. `Dizi[J] < Dizi[min]` ise `min = J`. Yalnızca en küçüğün yeri (indisi) saklanır, değeri değil.
    * İç döngüden sonra bir kez yer değiştirilir: `tmp = Dizi[I]`, `Dizi[I] = Dizi[min]`, `Dizi[min] = tmp`.
  - Örnekte ilk turda `min` sırasıyla 1, 2 ve 7 olur. 7. gözdeki 0 birinci gözle yer değiştirir: `0 1 3 2 4 8 5`.
  - Selection sort'ta en kötü durumda da en iyi durumda da N^2 karşılaştırma yapılır. Ama dış döngünün her turunda yalnızca bir yer değiştirme yapılır.
- Insertion sort (yalnızca mantığı)
  - Kağıt oyununda eldeki kartları sıralamaya benzer: elinizde 5 ve 8 varken 7 gelirse onu araya sokarsınız.
  - Her eleman, sıralanmış kısımda olması gereken yere yerleştirilir. Örnek (altyazıda `5 2 3 1 4 7 8` geçer, söylenen ara dizilere göre büyük olasılıkla `5 3 2 1 4 7 8`): önce 3 ve 5 sıralanır, sonra 2, sonra 1 araya yerleştirilir. En sonda 4 ile 5'in yerinin değiştirilmesi kalır.
  - Hoca, insertion sort'un bir sonraki derse kadar kaynaklara bakmadan kendi başına çözülmesini istedi.

Hocanın Özellikle Vurguladığı Kısımlar

- Algoritma geliştirmenin yolu tekrardır. Tekrar ettikçe beyin bu şekilde düşünmeye alışır ve ilk başta zor gelen tasarımlar kolaylaşır.
- Hoca soruları tahtada öğrencilerle birlikte, hazır bir çözümden bakmadan çözer. Bu yüzden zaman zaman küçük hatalar olabilir. Fark ederseniz uyarın. Sonradan fark ederseniz analiz yaparak mutlaka düzeltin.
- Bubble sort'ta iç döngüden önce ayrıca bir if kontrolü yapmaya gerek yoktur. Değişmeyecek elemanlar zaten if'in false koluna gider.
- Algoritma karmaşıklığı en yüksek üslü terimle ifade edilir. N^3 varsa N^2 de atılır.
- İyileştirilmiş bubble sort'ta `flag` her turun başında (iç döngüye girmeden) sıfırlanmalıdır. Aksi hâlde önceki turda yapılan bir değişiklik yeni turda değişiklik olmuş gibi görünür.
- Selection sort'ta iç döngüden sonra "en küçük zaten yerinde mi?" diye bir if konmaz. Bunu önceden bilemezsiniz. Eleman kendi üzerine yazılırsa bir zarar olmaz. Her turda kontrol yapmak yerine bu fazladan yer değiştirme kabul edilir.
- Yer değiştirmede (swap) hangi elemandan başladığınızın önemi yoktur. Birini geçici alana (`tmp`) alırsınız, onun yerine diğerini yazarsınız, sonra `tmp`'yi diğerinin yerine yazarsınız.
- Derste soruları çözmeye çalışın ve mikrofonla katılın. Az sayıda öğrencinin katılımı yeterli değildir.

Kısa Tekrar Notları

- Bubble sort: yan yana karşılaştır, gerekirse swap. Her tur en büyüğü sona iter.
- Bubble sort döngüleri: dış `1..N−1`, iç `1..N−I`.
- Bubble sort: en kötü N^2. İyileştirmesiz en iyi durum da N^2.
- İyileştirilmiş bubble sort: dış döngü while, `flag` ile "bu turda değişiklik oldu mu?" kontrolü. En iyi durum N.
- Karmaşıklık en yüksek üslü terimle yazılır.
- Selection sort: her turda en küçüğün indisini bul (`min`), turun sonunda bir kez swap.
- Selection sort: en kötü ve en iyi durumda N^2 karşılaştırma, tur başına bir swap.
- Insertion sort: her elemanı sıralı kısımdaki yerine yerleştir (kart dizme mantığı).

Detaylı Açıklamalar

Ders, önceki derste başlanan bubble sort'un tekrarıyla başlar. İç döngü sınırının `N-I` olması ve dış döngünün N−1 kez dönmesi bir örnek üzerinde gösterilir. Ardından bubble sort'un en kötü ve en iyi durum karmaşıklığı tartışılır ve en iyi durum için `flag` değişkeniyle bir iyileştirme yapılır. Aradan sonra selection sort'un ilkel ve iyileştirilmiş versiyonları birlikte çizilir. Son olarak insertion sort'un mantığı kart dizme örneğiyle anlatılır.

Bubble sort: neden her turda bir eleman daha az karşılaştırılır?

İlk turda 9 her karşılaştırmada bir sağa geçer: `9 1` → `1 9`, `9 3` → `3 9` ve böyle devam ederek dizinin sonuna ulaşır. İkinci turda 8 aynı şekilde sona doğru ilerler ve 9'un hemen soluna yerleşir. İkinci turda 8 ile 9'u karşılaştırmaya gerek yoktur, çünkü 9 ilk turda yerine oturmuştur. Bu yüzden I. turda iç döngü `N-I`'de durur.

Sekiz elemanlı örnekte yedinci turun sonunda `1 2 3 4 6 7 8 9` elde edilir. Sondaki yedi eleman yerine oturduğu için ilk eleman da doğru yerdedir. Sekizinci tura gerek yoktur.

Karmaşıklık nasıl ifade edilir?

Bir öğrenci en kötü durumdaki karşılaştırma sayısını $(N-1)\cdot(N-1)!$ olarak tahmin etti. Hoca bunun çok fazla olduğunu, öyle olsaydı bilgisayarların başa çıkamayacağını söyledi. Doğru ifade $(N-1)\cdot(N-I)$ şeklindedir. Genelleştirildiğinde küçük üslü terimler atılır ve N^2 karmaşıklığı elde edilir. Hoca bunu "N kez dönülen tur × her turda yaklaşık N karşılaştırma" diye özetledi.

Dizi zaten sıralı verilse bile ilk tasarım bütün karşılaştırmaları yapar. Bu yüzden en iyi durum da N^2'dir. Tek fark, hiçbir yer değiştirme yapılmamasıdır.

İyileştirilmiş bubble sort'ta `flag`

Öğrencilerden gelen fikirle şu tasarım ortaya çıktı:

1. `flag = 1` yapılır. Dış döngü `while flag = 1`.
2. Her turun başında `flag = 0` yapılır.
3. İç döngüde bir yer değiştirme olursa `flag = 1` yapılır.
4. Tur sonunda `flag` hâlâ 0 ise hiçbir eleman yer değiştirmemiştir. Dizi sıralıdır ve while biter.

Sıralı dizide ilk turda N−1 karşılaştırma yapılır ve hiç yer değiştirme olmaz. Döngü ilk turda biter. Bu yüzden en iyi durum karmaşıklığı N^2'den N'e iner. Bir öğrencinin "yine N^2" cevabı yanlıştı. Doğru cevabı bir başka öğrenci açıkladı: tek bir tur atılır ve tur N mertebesinde karşılaştırma yapar. En kötü durumda bu iyileştirmenin bir etkisi yoktur.

Hoca, bu iyileştirilmiş versiyonun akış diyagramını öğrencilerin kendilerinin çizmesini istedi.

Selection sort: ilkel ve iyileştirilmiş versiyon

İlkel versiyonda I. eleman, sağındaki her elemanla karşılaştırılır ve daha küçük bir eleman görüldüğü anda yer değiştirilir. `5 1 3 2 4 8 0` dizisinin ilk turunda önce 5 ile 1, sonra 1 ile 0 yer değiştirir: `0 5 3 2 4 8 1`. İkinci turda 5 ile 3, 3 ile 2, 2 ile 1 yer değiştirir. Bir turda birden fazla yer değiştirme olur. En büyük eleman baştaysa en kötü durumda bir turda N−1 yer değiştirme yapılır. Bu kötü bir tasarımdır.

İyileştirilmiş versiyonda iç döngü yalnızca o ana kadarki en küçük elemanın indisini (`min`) günceller. Tur bittiğinde en küçük eleman sıradaki konuma tek bir yer değiştirmeyle getirilir. Örnekte ilk turda `min = 1` ile başlanır, 1 bulununca `min = 2`, 0 bulununca `min = 7` olur. Sonra 1. ve 7. gözler yer değiştirir.

İkinci turda `I = 2` ve `min = 2` ile başlanır. İç döngü `I+1`'den, yani 3'ten başlar. Kalan elemanların hiçbiri 1'den küçük olmadığı için `min` değişmez. Ardından 2. göz kendi üzerine yazılır. Bir öğrenci bunu önlemek için `min ≠ I` kontrolü önerdi. Hoca, her turda bu kontrolü yapmanın gereksiz olduğunu ve fazladan bir yer değiştirmenin kabul edildiğini vurguladı.

En küçüğün değeri yerine yeri saklanır. Değeri de saklayan çözümler yanlış değildir, ama yeri saklamak yeterlidir ve yer değiştirmeyi doğrudan yapmayı sağlar.

Selection sort'ta en kötü durumda (ör. büyükten küçüğe sıralı dizi) N^2 karşılaştırma yapılır. En iyi durumda da (sıralı dizi) bütün karşılaştırmalar yapılır, yalnızca true koluna girilmez. Bu yüzden en iyi durum da N^2'dir. Kazanç, karşılaştırma sayısında değil, yer değiştirme sayısındadır.

Kontrol

Ders sonrasında iyileştirilmiş bubble sort ve iyileştirilmiş selection sort C'ye çevrilip `gcc -Wall` ile derlendi. `5 1 3 2 4 7 8 6` ve `5 1 3 2 4 8 0` dizileri doğru sıralandı. Sıralı 8 elemanlı dizide iyileştirilmiş bubble sort yalnızca 7 karşılaştırma yaptı.

```c
void bubble_iyilestirilmis(int dizi[], int N)
{
    int i=1,j,tmp,flag=1;
    while(i<=N-1 && flag==1)
    {
        flag=0;
        for(j=0;j<N-i;j++)
            if(dizi[j]>dizi[j+1])
            {
                tmp=dizi[j]; dizi[j]=dizi[j+1]; dizi[j+1]=tmp;
                flag=1;
            }
        i++;
    }
}

void selection(int dizi[], int N)
{
    int i,j,min,tmp;
    for(i=0;i<N-1;i++)
    {
        min=i;
        for(j=i+1;j<N;j++)
            if(dizi[j]<dizi[min])
                min=j;
        tmp=dizi[i]; dizi[i]=dizi[min]; dizi[min]=tmp;
    }
}
```

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.